

Road Network Navigation to Arbitrary Poses

Road Network Navigation to Arbitrary Poses

1. Course Content
2. Running the Road Network Navigation Function
 - 2.1 Start the Agent
 - 2.2 Start Sensors and Odometry
 - 2.3 Start the Road Network Navigation Node
 - 2.4 Arbitrary Pose Road Network Navigation
 - 2.4.1 Application Example
3. Source Code Analysis
 - 3.1 Analysis of route_bridge.py

1. Course Content

Based on navigation between waypoints, this combines the road network planning and navigation function to achieve navigation to any arbitrary pose.

[!NOTE]

When using `route_server` from navigation2 for road network navigation, it can only achieve navigation with a fixed orientation. After reaching the target waypoint, the robot will always face the direction of the edge in the road network. In practical applications, we need the robot to face a specified orientation at the target point to complete specific tasks. The road network navigation is only treated as an intermediate global path.

2. Running the Road Network Navigation Function

2.1 Start the Agent

- Start the terminal for the humble version of docker

```
start_muto_humble
```

2.2 Start Sensors and Odometry

- After starting docker, use the command to start the radar fusion node and odometry fusion.

```
ros2 launch yahboomcar_nav laser_odom_bringup_launch.py
```

2.3 Start the Road Network Navigation Node

- If the `ros2_kilted` docker container is not started, you need to start it first.

```
~/kilted_docker_start.sh
```

- Start the road network planning and navigation node.

```
ros2 launch road_net_route roadnet_nav2.launch.py
map:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/road_map.yaml
pose_map:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/road_map
roadnet_file:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/road_map.geojson
```

[!TIP]

Optional startup parameter descriptions

map The name of the grid map file to load, default is `map.yaml`.

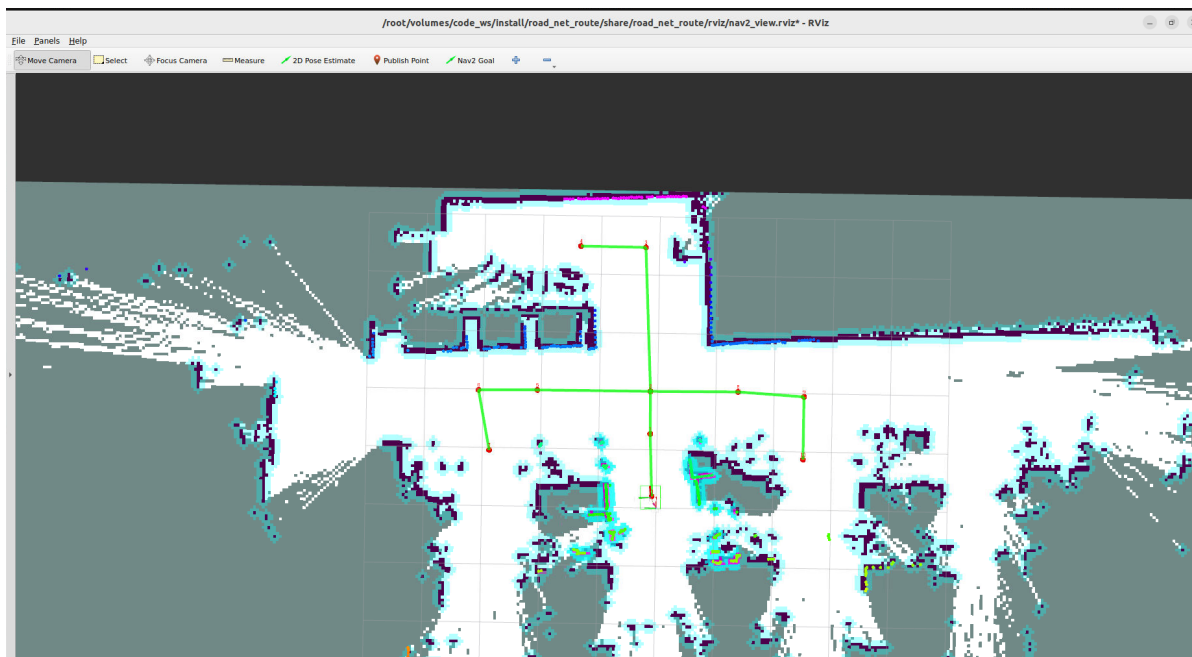
pose_map The name of the pose map file to load. When using `slam_toolbox` for pure localization as the global localization method, a pose map must be provided.

params_file Navigation parameter file, default is

```
os.path.join(get_package_share_directory('road_net_route'), 'config',
'nav2_kilted.yaml').
```

roadnet_file Road network description file, default is `/root/map/map.geojson`. A full file path must be provided here.

localization_mode Global localization method, default is `slam_toolbox`.



2.4 Arbitrary Pose Road Network Navigation

- Here we use road network navigation by publishing to the `/road_net_nav` topic in the terminal. This topic can accept a `PoseStamped` target pose. The robot will travel along the road network to the vicinity of the target pose, and then accurately adjust to the target pose. The format for publishing the topic is as follows:

```
ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  pose: {
    position: {x: xx, y: xx, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: xx, w: xx}
  }
}"
```

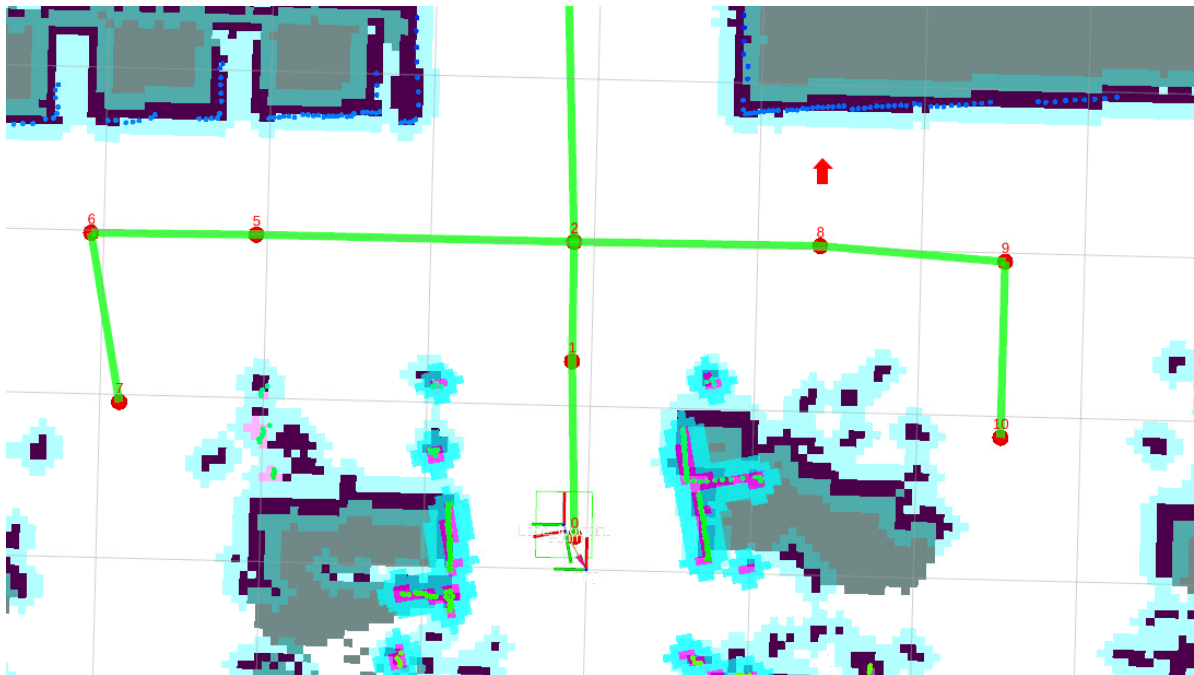
[!NOTE]

A pose consists of a position and an orientation. For a ground mobile robot, the position only has planar x, y coordinates, and the orientation is only rotation around its own z-axis. In the pose:

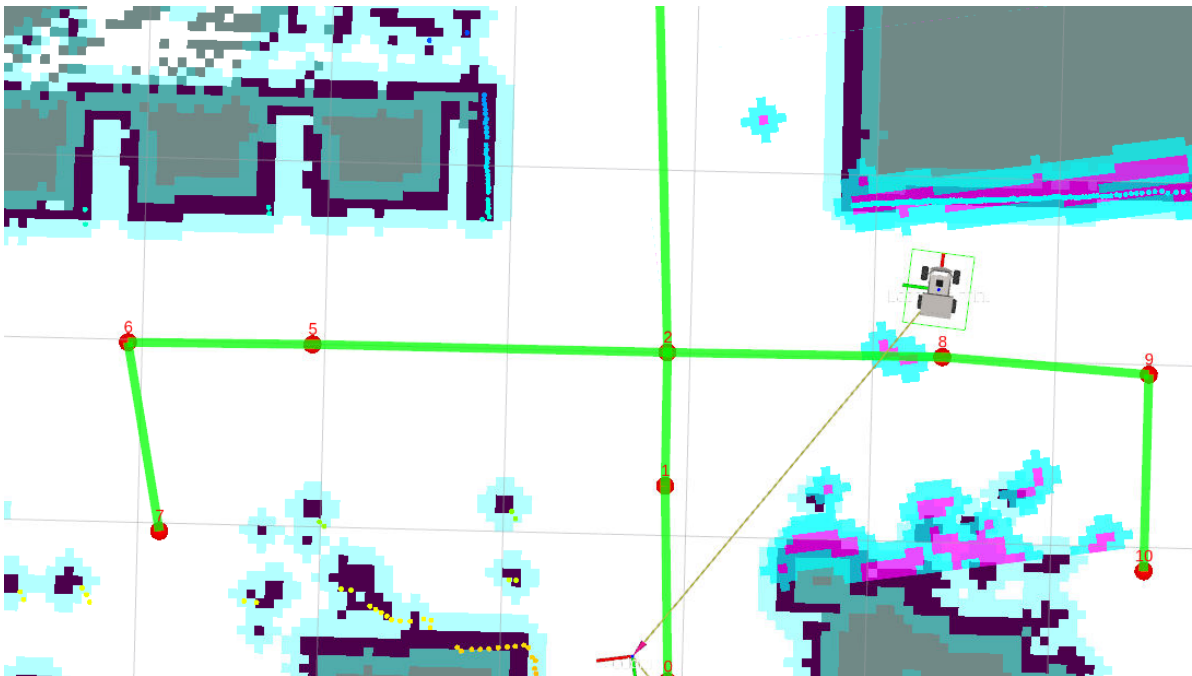
- **position: {x: xx, y: xx, z: 0.0}** is the position information.
- **orientation: {x: 0.0, y: 0.0, z: xx, w: xx}** is the body rotation information described by a quaternion.

2.4.1 Application Example

- For example, if we need the robot to travel only along the road network and then navigate to a position above waypoint 8 on the map with a specified orientation (as indicated by the arrow), we first need to know the coordinates of this pose in the global map.



- First, let's move the robot to this target pose.



- Use the following command to get the current coordinates of the robot on the map.

```
ros2 run tf2_ros tf2_echo map base_footprint
```

```

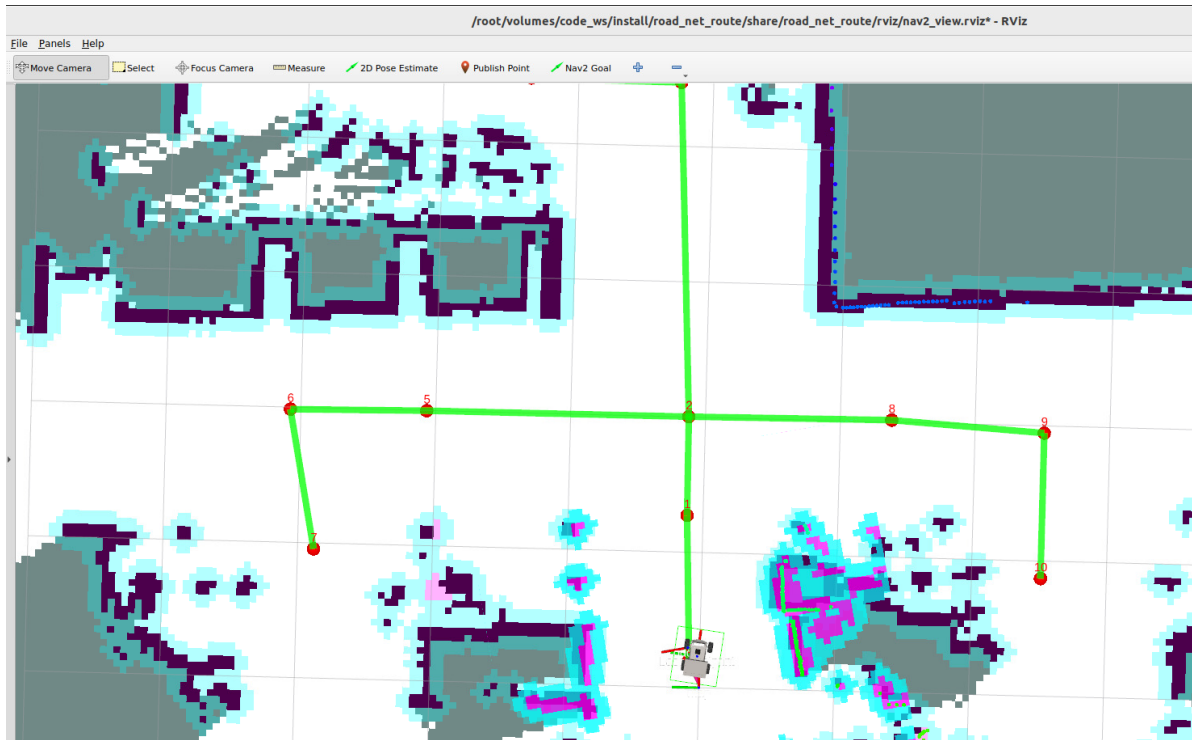
yahboom: ~
jetson@yahboom: ~ 70x19
- Rotation: in RPY (radian) [0.000, 0.000, -0.104]
- Rotation: in RPY (degree) [0.000, 0.000, -5.946]
- Matrix:
  0.995  0.104  0.000  2.383
 -0.104  0.995 -0.000 -1.352
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1763608206.239000000
- Translation: [2.383, -1.352, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.052, 0.999]
- Rotation: in RPY (radian) [0.000, 0.000, -0.104]
- Rotation: in RPY (degree) [0.000, 0.000, -5.959]
- Matrix:
  0.995  0.104  0.000  2.383
 -0.104  0.995 -0.000 -1.352
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
^C[INFO] [1763608207.095851031] [rclcpp]: signal_handler(signum=2)
jetson@yahboom:~$

```

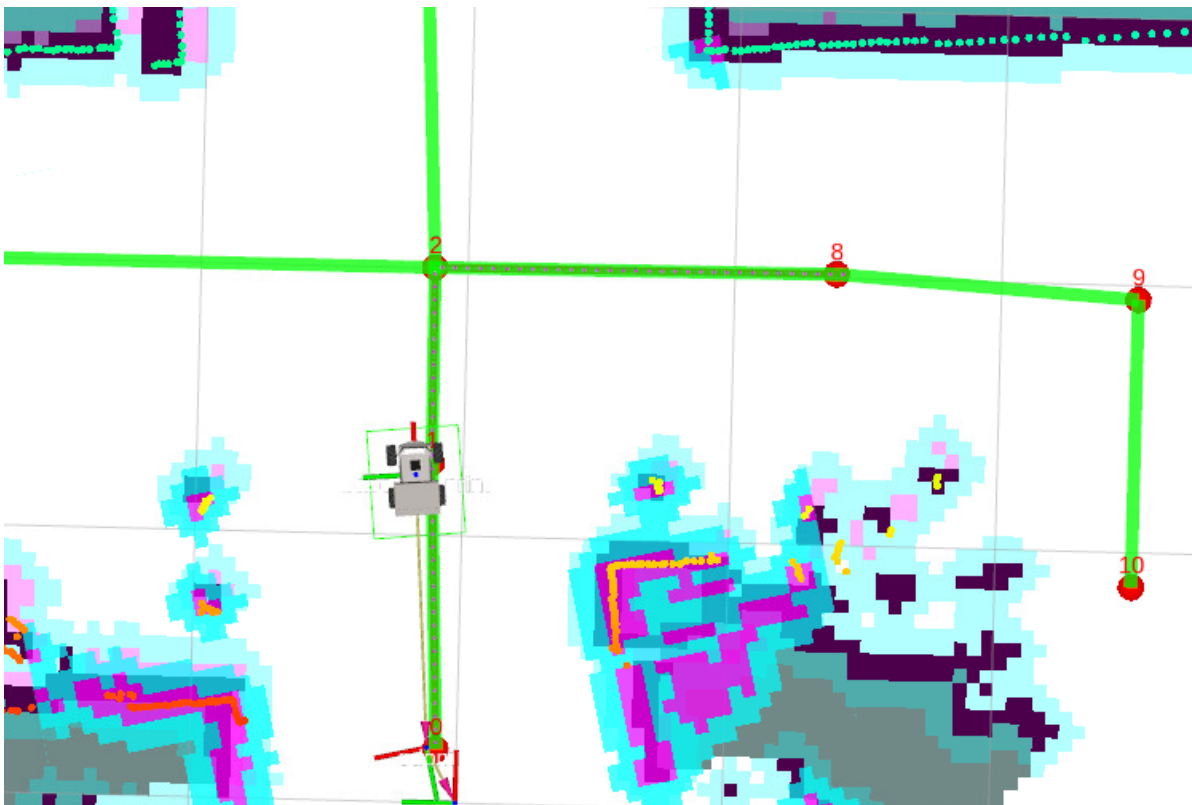
The `Translation` and `Rotation` are the pose of the robot in the global map.

- Then, let the robot return to its starting position and publish the target pose in the terminal.

```
ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  header: {frame_id: 'map'},
  pose: {
    position: {x: 2.383, y: -1.352, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: -0.052, w: 0.999}
  }
}"
```



- The robot will first travel along the road network to a waypoint near the target point.

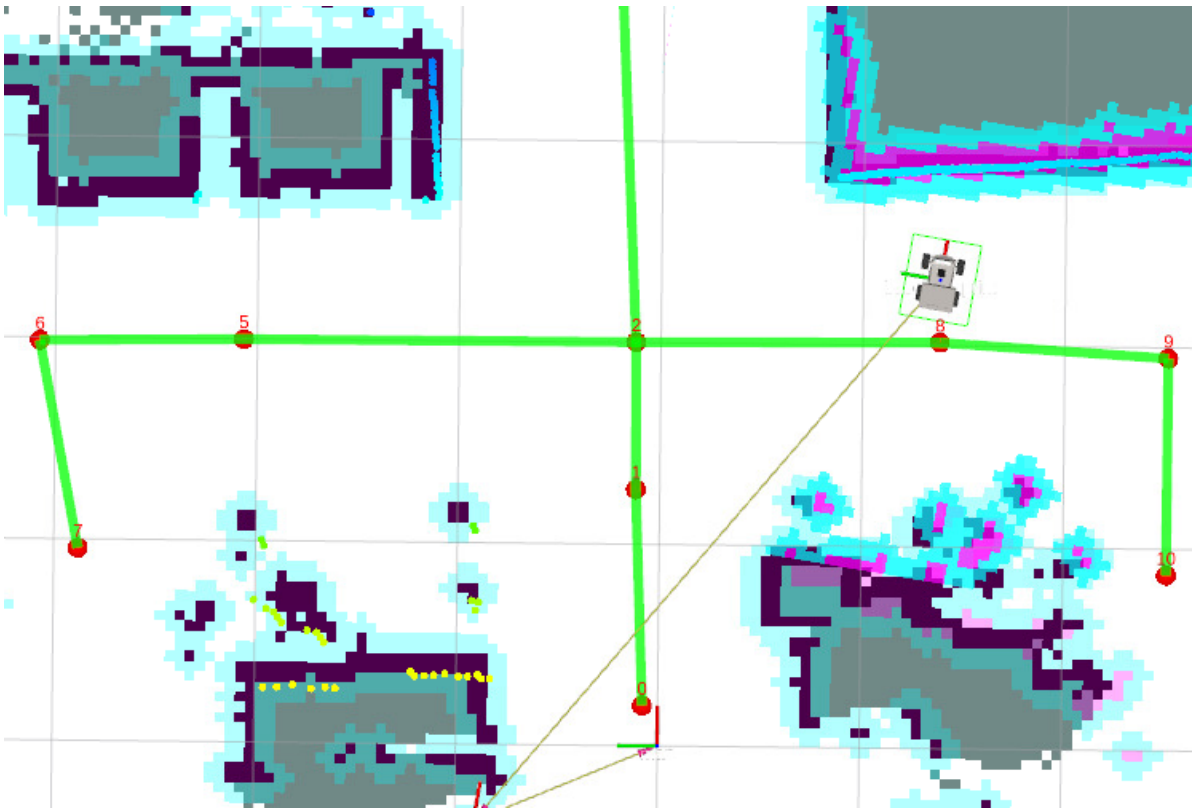


- Finally, it adjusts to the target pose, and the terminal prompts `navigation task completed`.

```

[controller_server]: Passing new path to controller.
[component_container_isolated-1] [INFO] [1763609230.200338386] [controller_server]: Reached the goal!
[component_container_isolated-1] [INFO] [1763609230.220298226] [bt_navigator]: Goal succeeded
[route_bridge-4] [INFO] [1763609230.274454384] [route_bridge]: navigation task completed.

```



3. Source Code Analysis

- Path to the implementation source code:

```
~/code_ws/src/road_net_route/road_net_route/route_bridge.py
```

3.1 Analysis of route_bridge.py

- In the `__init__` function of the `Route_Bridge` class, a subscriber to the `road_net_nav` topic is created. It receives `PoseStamped` data from the `road_net_nav` topic and directly calls `getAndTrackRoute` to initiate road network navigation:

```
self.road_net_nav_sub= self.create_subscription(PoseStamped, "road_net_nav",
self.nav_callback, 10)
```

- `PoseStamped`: Message type, containing target position and orientation information.
- `"road_net_nav"`: Topic name, used to send the navigation target pose.
- `self.nav_callback`: Callback function, handles the core logic of the navigation command.
- `10`: Message queue size. If the callback processing is slow, it can cache up to 10 messages to avoid message loss.

- Step 1: Get the robot's current pose (TF transform): Using `tf2_ros`'s `Buffer` and `TransformListener`, get the coordinate transformation from the robot's base coordinate system (`base_footprint`) to the global map coordinate system (`map`), thus obtaining the robot's current pose on the map.
- Step 2: Construct the robot's current pose: Encapsulate the result of the TF transform (translation + rotation) into a `PoseStamped` type `start_pose`, which serves as the **starting pose** for navigation.
- Step 3: Convert pose to road network node ID: This is the **core conversion logic of road network navigation**:
 - Call the `__find_nearest_node` method to match the **start pose** and **goal pose** to the nearest node IDs in the road network (via KDTree nearest neighbor search).
 - Call the `BasicNavigator`'s `getAndTrackRoute` interface, passing the start node ID and goal node ID, to initiate navigation based on road network nodes.
- Step 4: Final posture adjustment: Road network navigation only ensures that the robot reaches a waypoint near the target pose. The `goToPose` interface is used to have the robot accurately adjust to the target posture.

```
def nav_callback(self, msg: PoseStamped):
    ''' Arbitrary Pose Road Network Navigation Callback Function'''

    transform = self.tf_buffer.lookup_transform("map", "base_footprint",
rcipy.time.Time(), rcipy.duration.Duration(seconds=5.0))
    if transform is None:
        self.get_logger().info(Fore.RED+"Failed to get TF transform
"+Fore.RESET)
        return
    else:
        start_pose = PoseStamped()
        start_pose.header = transform.header
        start_pose.pose.position.x = transform.transform.translation.x
        start_pose.pose.position.y = transform.transform.translation.y
        start_pose.pose.orientation = transform.transform.rotation
        self.get_logger().info(Fore.CYAN + f"current:
{transform.transform.translation}"+Fore.RESET)

        if self.replace2id:
            nearest_start_node_id, nearest_start_distance =
self.__find_nearest_node(start_pose.pose.position.x, start_pose.pose.position.y)
            nearest_goal_node_id, nearest_goal_distance =
self.__find_nearest_node(msg.pose.position.x, msg.pose.position.y)
            self.get_logger().info(Fore.CYAN + f"nearest_start_node_id:
{nearest_start_node_id}, distance:{nearest_start_distance}"
f"\nnearest_goal_node_id:
{nearest_goal_node_id}, distance:{nearest_goal_distance}"
+Fore.RESET)

            route_tracking_task =
self.navigator.getAndTrackRoute(start=nearest_start_node_id, goal=nearest_goal_no
de_id)
        else:
            route_tracking_task =
self.navigator.getAndTrackRoute(start=start_pose, goal=msg, use_start=False)
            task_canceled = False
            last_feedback = None
            follow_path_task = RunningTask.NONE
```

```

        while not self.navigators.isTaskComplete(task=route_tracking_task):
            feedback = self.navigators.getFeedback(task=route_tracking_task)
            while feedback is not None:
                if not last_feedback or (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id !=
last_feedback.next_node_id):
                    self.get_logger().info('Passed node ' +
str(feedback.last_node_id) +
                    ' to next node ' + str(feedback.next_node_id) +
                    ' along edge ' + str(feedback.current_edge_id) +
                    '.')

                last_feedback = feedback
                if feedback.rerouted: # or follow_path_task ==
RunningTask.None
                    self.get_logger().info('Passing new route to
controller!')
                    follow_path_task =
self.navigators.followPath(feedback.path)
                    feedback =
self.navigators.getFeedback(task=route_tracking_task)
                    if self.navigators.isTaskComplete(task=follow_path_task):
                        self.get_logger().info('Controller or waypoint follower
server completed its task!')
                        self.navigators.cancelTask()
                        task_canceled = True
                    while not self.navigators.isTaskComplete(task=follow_path_task) and
not task_canceled:
                        time.sleep(0.1)

                result = self.navigators.getResult()
                if result == TaskResult.SUCCEEDED:
                    if self.__adjust_pose(msg):
                        self.get_logger().info(Fore.GREEN+"navigation task
completed."+Fore.RESET)

                        self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
                        elif result == TaskResult.CANCELED:
                            self.get_logger().info(Fore.YELLOW+"navigation task
canceled."+Fore.RESET)
                            elif result == TaskResult.FAILED:
                                self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)

                                self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
                                else:
                                    self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)

                                    self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

def __adjust_pose(self, final_pose)->bool:
    '''Adjust to target posture'''
    self.get_logger().info("adjusting final orientation...")
    final_orientation_task = self.navigators.goToPose(final_pose)
    while not self.navigators.isTaskComplete(task=final_orientation_task):
        time.sleep(0.1)

    orientation_result = self.navigators.getResult()

```

```

        if orientation_result == TaskResult.SUCCEEDED:
            return True
        else:
            self.get_logger().info("final orientation adjustment failed.")
            return False

def __find_nearest_node(self, x, y):
    '''Use KDTree to find the nearest node'''
    distance, index = self.tree.query([x, y])
    nearest_node_id = self.node_ids[index]
    return nearest_node_id, distance

```

- The `__load_roadnet_nodes` method is used during the program initialization phase to parse the **GeoJSON format** road network file, extract the `Point` type nodes (topological nodes of the road network), store the node coordinates and IDs, and build a `KDTree` index.

```

def __load_roadnet_nodes(self):
    '''Load road network nodes for KDTree search'''
    with open(self.road_net_file, 'r') as f:
        data = json.load(f)

    self.node_coords = []    # [[x, y], ...]
    self.node_ids = []      # Corresponding node ID

    for feat in data['features']:
        if feat['geometry']['type'] == 'Point':
            x, y = feat['geometry']['coordinates']
            self.node_coords.append([x, y])
            self.node_ids.append(feat['properties'].get("id", None))

    self.tree = KDTree(self.node_coords)
    self.get_logger().info(f"Loaded {len(self.node_coords)} nodes into KDTree")

```